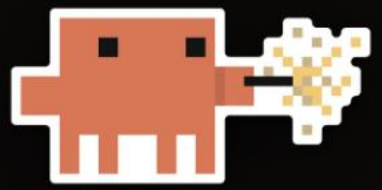
// Píldoras de Programación · Curso Anthropic

## Claude Code in Action IV

# Skills

Una carpeta con instrucciones que le enseña a Claude a hacer una tarea específica **a tu manera**, y que Claude carga solo cuando la necesita.

Claude Code · SKILL.md · carga progresiva · Skills vs MCP vs Hooks



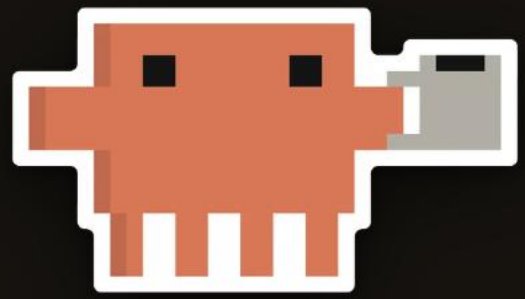
# Un skill es básicamente una carpeta con instrucciones



mi-skill/

instrucciones + recursos

se carga cuando hace falta



Claude, haciendo la tarea **a tu manera**

### Una carpeta con instrucciones

Le enseña a Claude a hacer **una tarea específica**, con tus reglas, tus pasos y tu estilo.

### Claude la carga solo cuando la necesita

No vive todo el tiempo en el contexto: entra cuando la tarea lo amerita.

### ...o la invocas tú a mano

Escribes `/nombre-skill` y se ejecuta. Los antiguos slash commands de `.claude/commands/` hoy son skills: la misma cosa, el mismo archivo.

// 02 · Definición oficial

# Capacidades modulares que extienden la funcionalidad de Claude

Cada Skill **empaqueta** instrucciones, metadatos y recursos opcionales (scripts, plantillas) que Claude usa **automáticamente cuando son relevantes**.

01

## Instrucciones

El cuerpo Markdown del `SKILL.md` : los pasos, las reglas y el estilo que Claude sigue al hacer la tarea.

```
## 1. Estilo de código
- Sigue las convenciones PEP 8.
- Utiliza nombres descriptivos.
```

02

## Metadatos

El frontmatter YAML: `name` y `description` . Con eso Claude decide **cuándo** usar la skill.

```
---
name: retos-python
description: Úsala cuando…
---
```

03

## Recursos opcionales

Scripts, plantillas y documentación de apoyo que viven en la misma carpeta y se leen solo si hacen falta.

```
references/   docs
scripts/      código
assets/       plantillas
```



# Estructura mínima: un directorio con un SKILL.md

Frontmatter YAML + contenido Markdown. El campo `description` determina cuándo Claude invoca tu Skill.

tree mi-skill

```
mi-skill/
├── SKILL.md          ← obligatorio
├── REFERENCE.md      ← opcional
└── scripts/          ← opcional
    └── fill_form.py

$ cat mi-skill/SKILL.md
---
name: mi-skill
description: Úsala cuando…
---
# Instrucciones
1. …
```

### SKILL.md — obligatorio

Frontmatter ( `name` , `description` ) + las instrucciones en Markdown. Es el único archivo que no puede faltar.

### REFERENCE.md — opcional

Material de consulta más largo que no quieres cargar siempre.

### scripts/ — opcional

Código ejecutable que la skill puede correr, por ejemplo `fill_form.py` .

# El description decide cuándo Claude invoca tu Skill

retos-python/SKILL.md

```
---
name: retos-python
description: Úsala cuando el usuario pida crear
  un reto de programación en Python para
  la academia
---

# Instrucciones
1. Define el nivel (básico, intermedio…)
2. Escribe el enunciado con ejemplo
3. Agrega 3 tests de pytest
4. Entrega la solución al final
```

**`name` · el identificador**

Minúsculas y guiones, máximo 64 caracteres. Si lo omites, toma el nombre de la carpeta.

**`description` · cuándo usarla**

Es lo que Claude lee para decidir si la skill aplica a la tarea actual. Máximo 1024 caracteres: corta y específica.

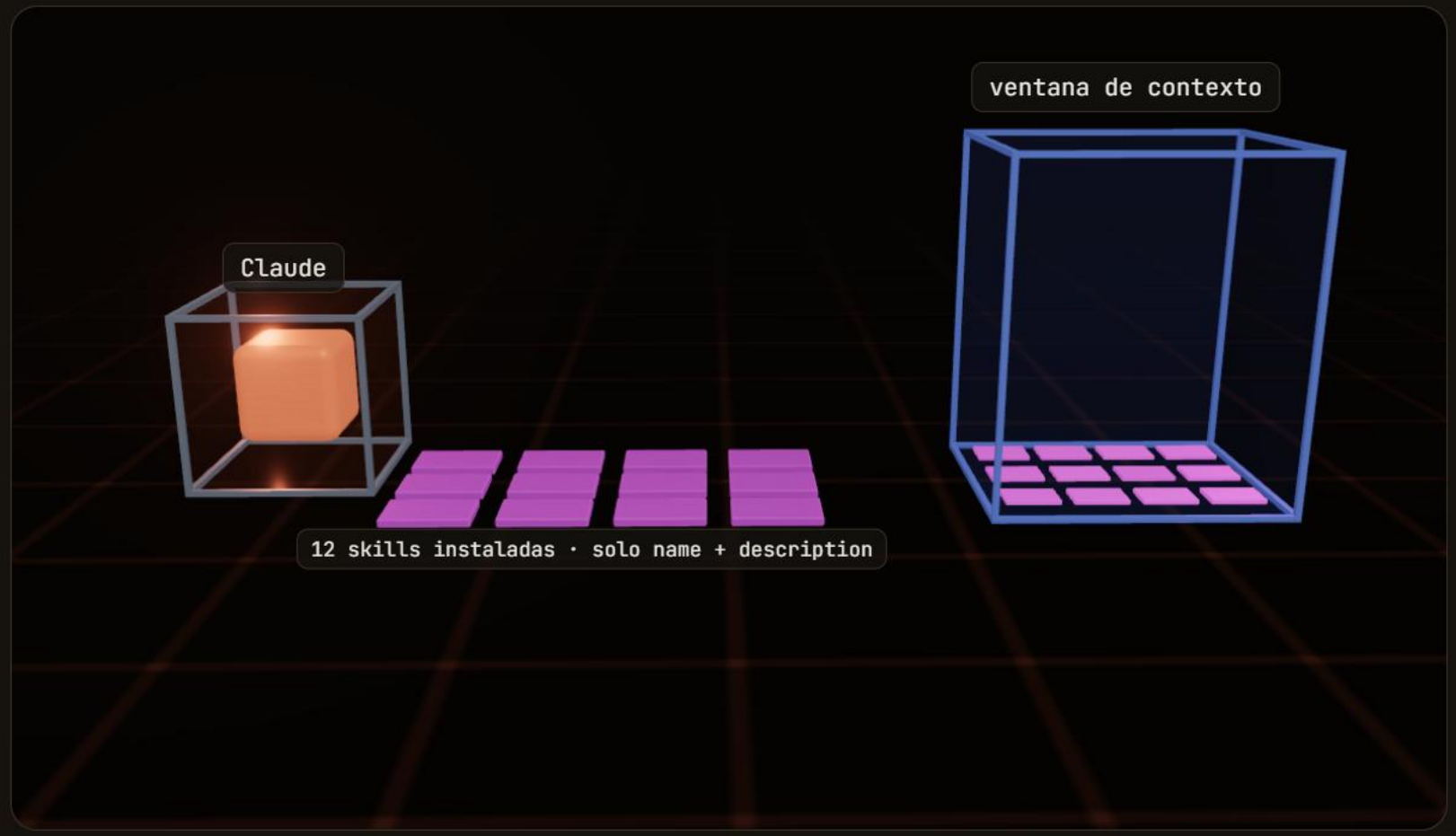
**`# Instrucciones` · qué hacer**

El Markdown que Claude sigue. También la puedes llamar a mano con `/retos-python`.

Existen más campos opcionales (argument-hint, disable-model-invocation, allowed-tools, model, context) · los vemos en otra entrega.

// 05 · Cómo funciona

# Carga progresiva: Claude solo lee lo que necesita, por niveles




| NIVEL 1 | NIVEL 2 | NIVEL 3 |
|---|---|---|
| 12 skills | — | — |
| name + description | SKILL.md completo | archivos extra |

CONTEXTO OCUPADO · ilustrativo

**1.200** tokens aprox.

PASO ACTUAL

**Nivel 1.** Claude siempre ve solo el **name** y la **description** de cada skill instalada: 12 fichas pequeñas entran al contexto. Si hay muchísimas, Claude Code recorta descripciones para caber en el presupuesto, empezando por las menos usadas.



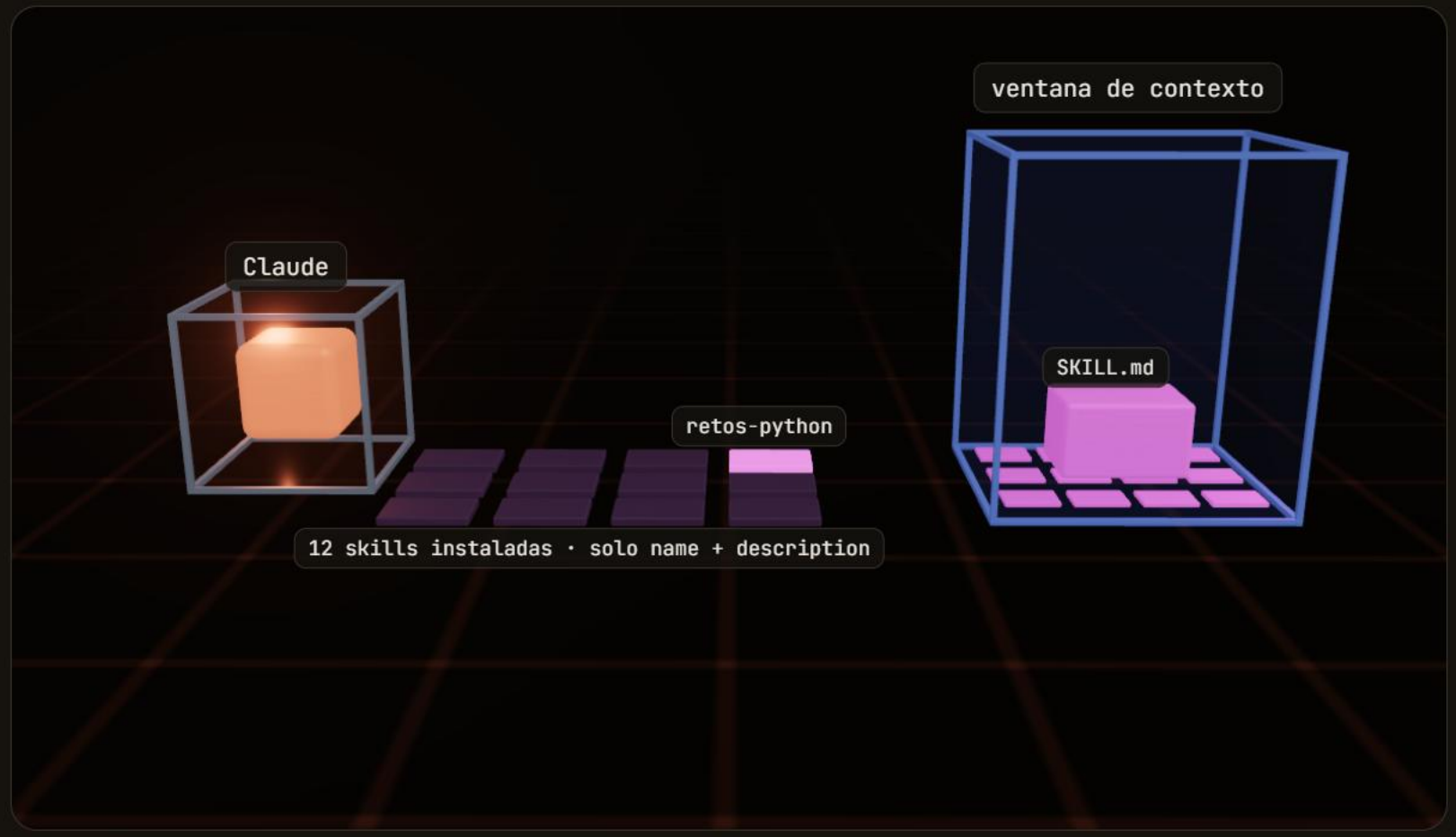
tokens simulados · ilustrativos



LOGO PÍLDORAS PENDIENTE

// 05 · Cómo funciona

# Carga progresiva: Claude solo lee lo que necesita, por niveles




NIVEL 1
**12** skills
name + description

NIVEL 2
**1**
SKILL.md completo

NIVEL 3
—
archivos extra

CONTEXTO OCUPADO · ilustrativo
**4.200** tokens aprox.

PASO ACTUAL

**Nivel 2.** Claude decide que **retos-python** es relevante para la tarea actual y carga su **SKILL.md completo** en el contexto. Las otras 11 siguen sin cargarse.



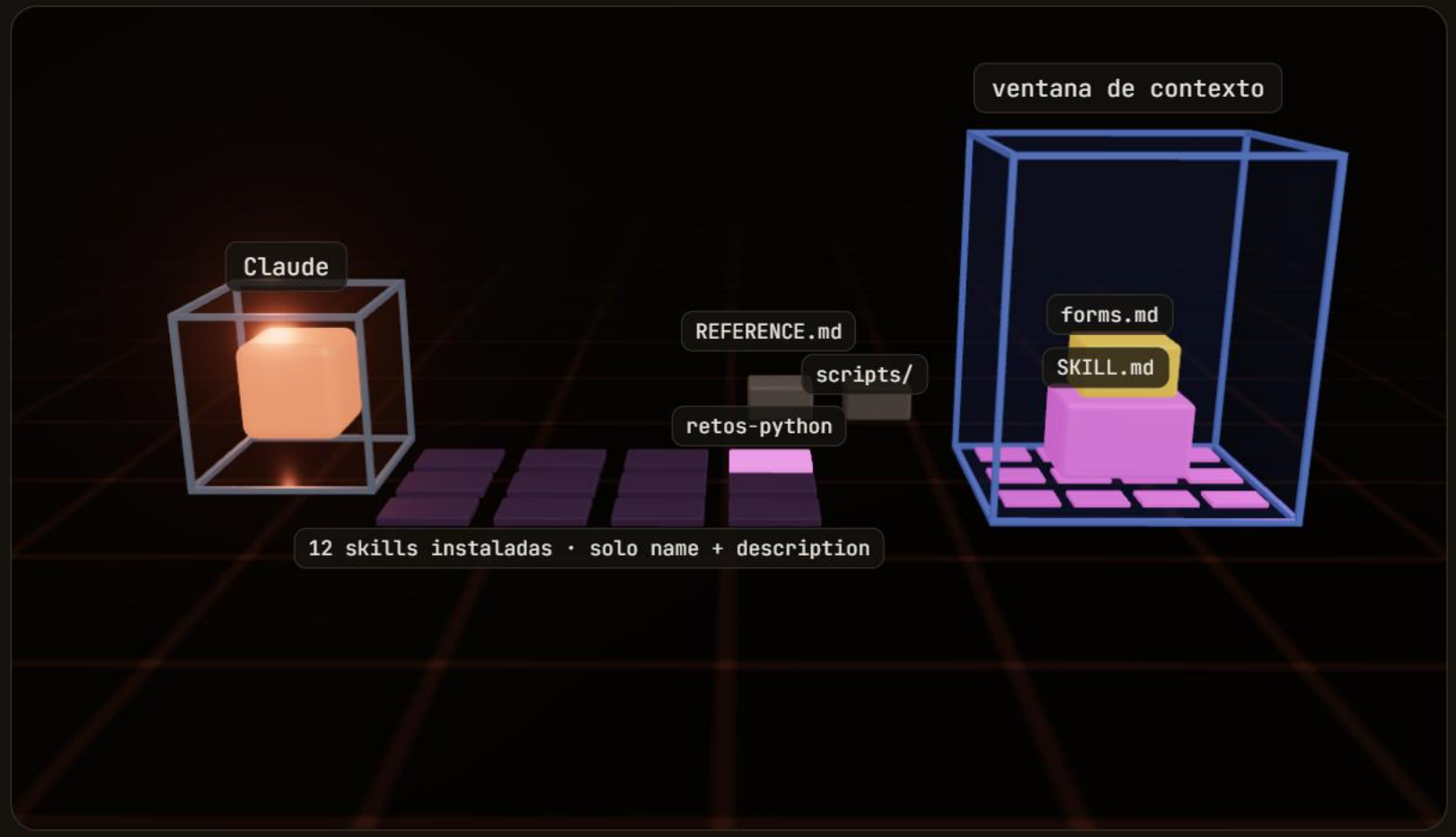
tokens simulados · ilustrativos



LOGO PÍLDORAS PENDIENTE

// 05 · Cómo funciona

# Carga progresiva: Claude solo lee lo que necesita, por niveles




NIVEL 1
**12** skills
name + description

NIVEL 2
**1**
SKILL.md completo

NIVEL 3
**1**
archivos extra

CONTEXTO OCUPADO · ilustrativo
**6.200** tokens aprox.

PASO ACTUAL

**Nivel 3.** Los archivos extra se leen solo si hacen falta: en la skill de PDF las instrucciones de formularios viven en **forms.md** y Claude lo lee únicamente cuando va a llenar un formulario. REFERENCE.md y scripts/ se quedan afuera.



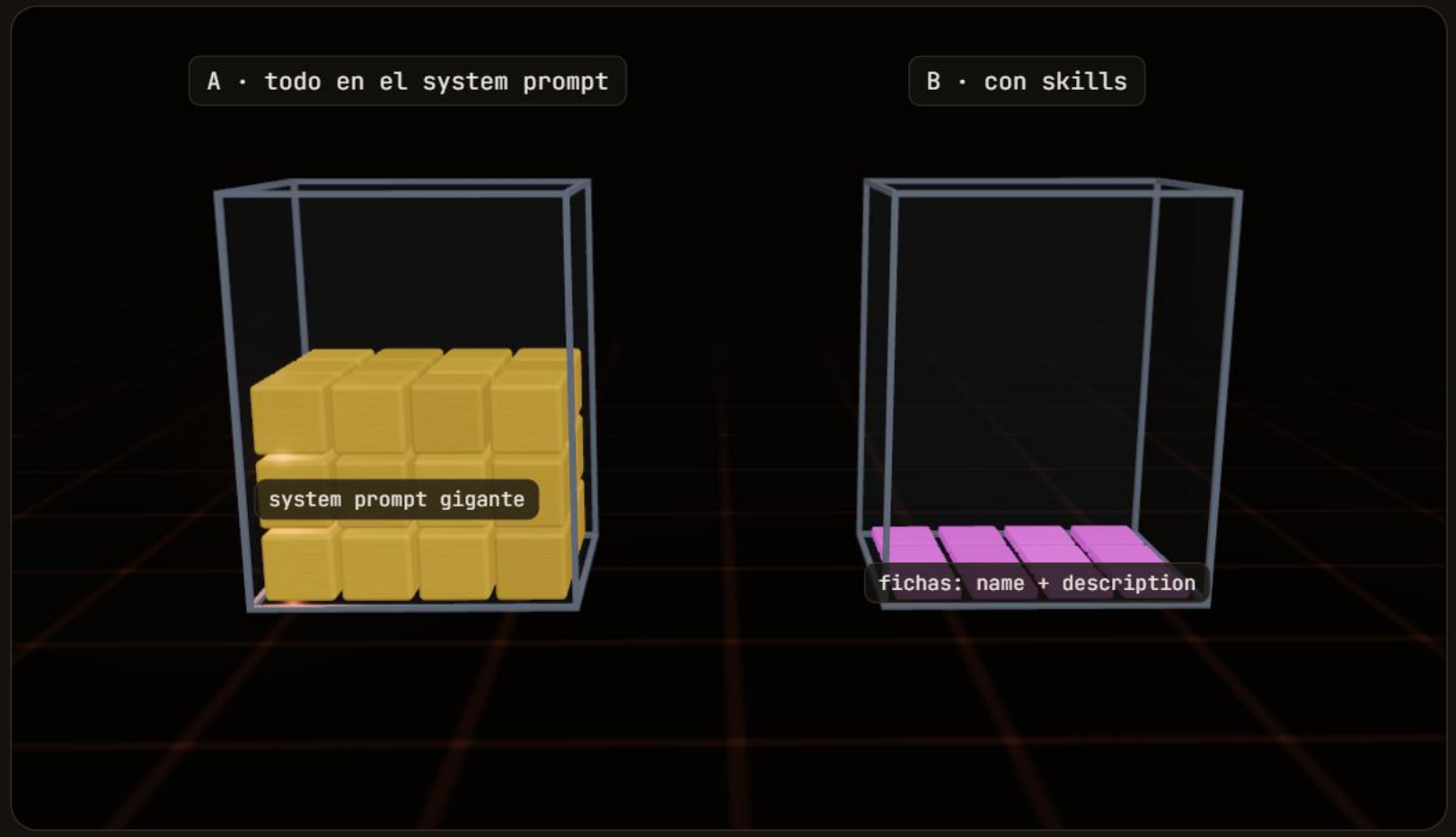
tokens simulados · ilustrativos



LOGO PÍLDORAS PENDIENTE

// 06 · Por qué importa

# Muchas skills instaladas sin saturar la ventana de contexto




TODO EN EL SYSTEM PROMPT

60 %

del contexto

CON SKILLS

5 %

del contexto

LEYENDA

- instrucciones de todos los temas
- skills (fichas / SKILL.md)
- tu conversación y tu código
- se desborda

PASO ACTUAL

**Paso 1.** En A, las instrucciones de todos los temas (PDF, Excel, Git, retos…) se cargan de entrada y ya ocupan buena parte del contexto. En B solo entran las fichas: name + description de cada skill, una franja mínima.



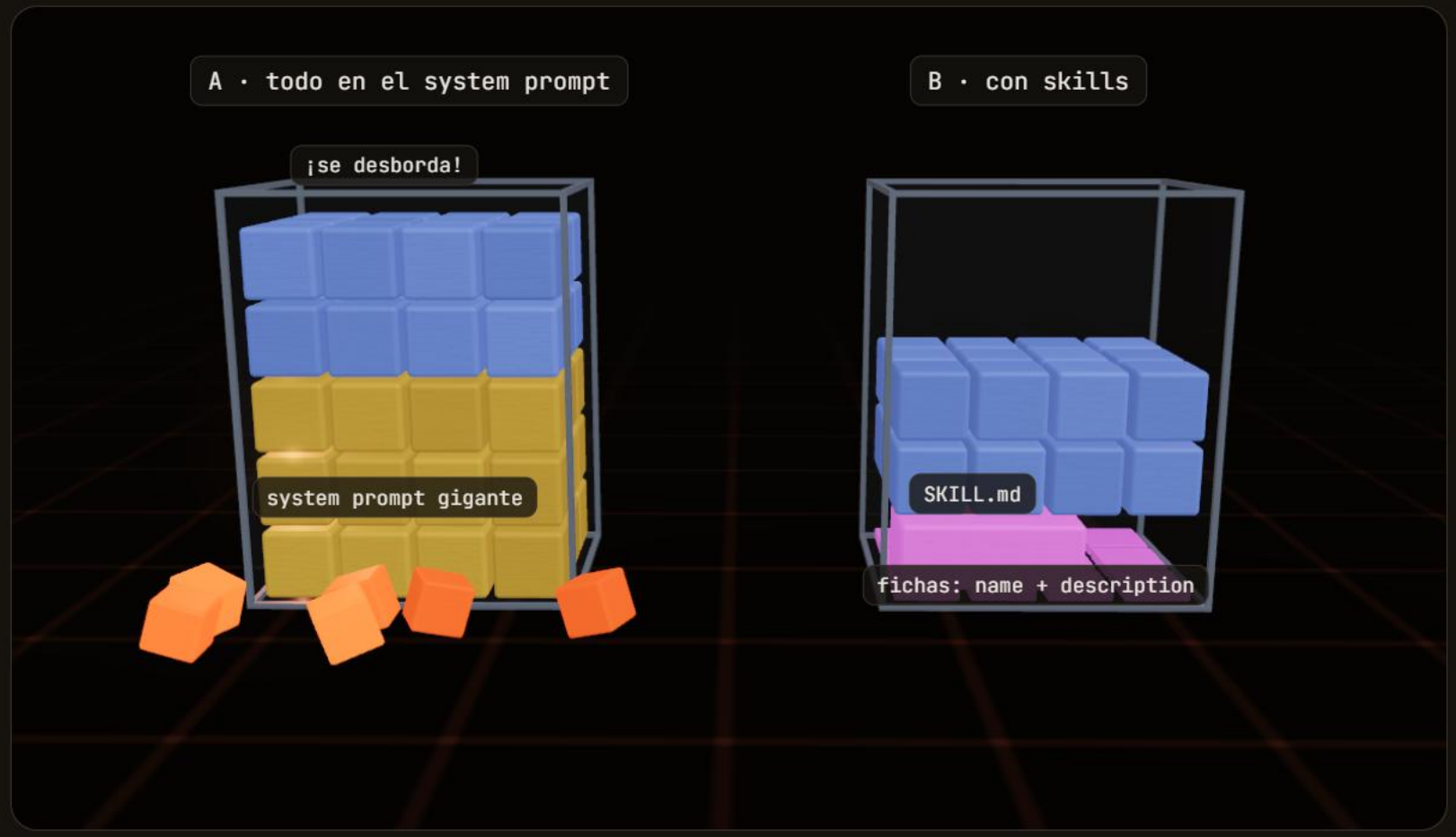
volúmenes simulados · ilustrativos



LOGO PÍLDORAS PENDIENTE

// 06 · Por qué importa

# Muchas skills instaladas sin saturar la ventana de contexto




TODO EN EL SYSTEM PROMPT
**100 %**
del contexto

CON SKILLS
**45 %**
del contexto

LEYENDA
- instrucciones de todos los temas
- skills (fichas / SKILL.md)
- tu conversación y tu código
- se desborda

PASO ACTUAL

**Paso 2.** Llega tu conversación y tu código. En B, Claude carga además el SKILL.md que necesita, y sigue sobrando espacio. En A la ventana se llena y se desborda: meter todo en un system prompt la agota mucho más rápido.



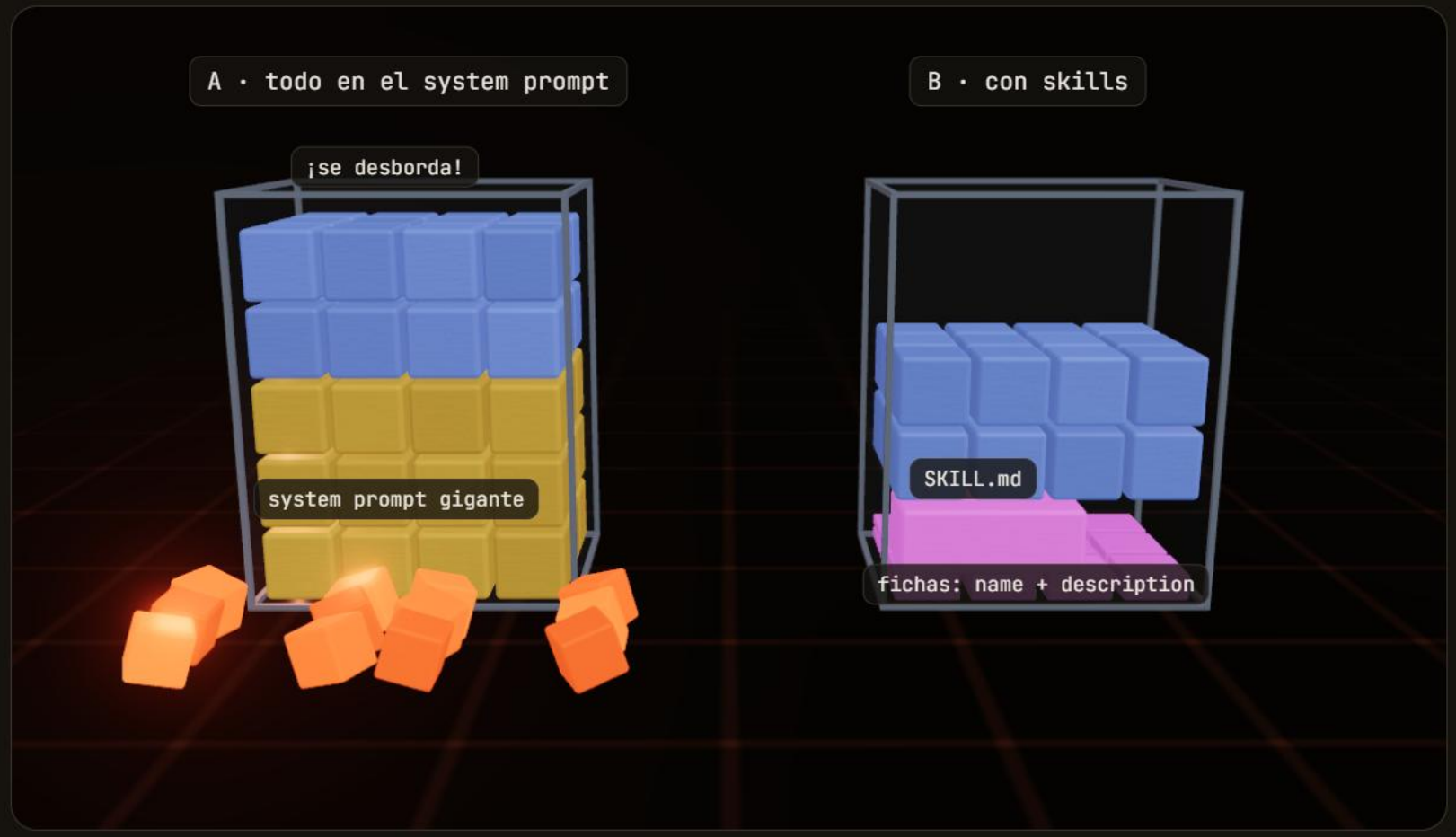
volúmenes simulados · ilustrativos



LOGO PÍLDORAS PENDIENTE

// 06 · Por qué importa

# Muchas skills instaladas sin saturar la ventana de contexto




TODO EN EL SYSTEM PROMPT
**100** %
del contexto

CON SKILLS
**48** %
del contexto

LEYENDA
- instrucciones de todos los temas
- skills (fichas / SKILL.md)
- tu conversación y tu código
- se desborda

PASO ACTUAL

**Paso 3.** Instalas 12 skills más. En B la franja de fichas apenas crece. En A no caben: así puedes tener muchas skills instaladas sin saturar la ventana de contexto.



volúmenes simulados · ilustrativos



LOGO PÍLDORAS PENDIENTE

// 07 · Dónde se guardan

# Tres niveles, tres alcances: organización, tu máquina, el repo



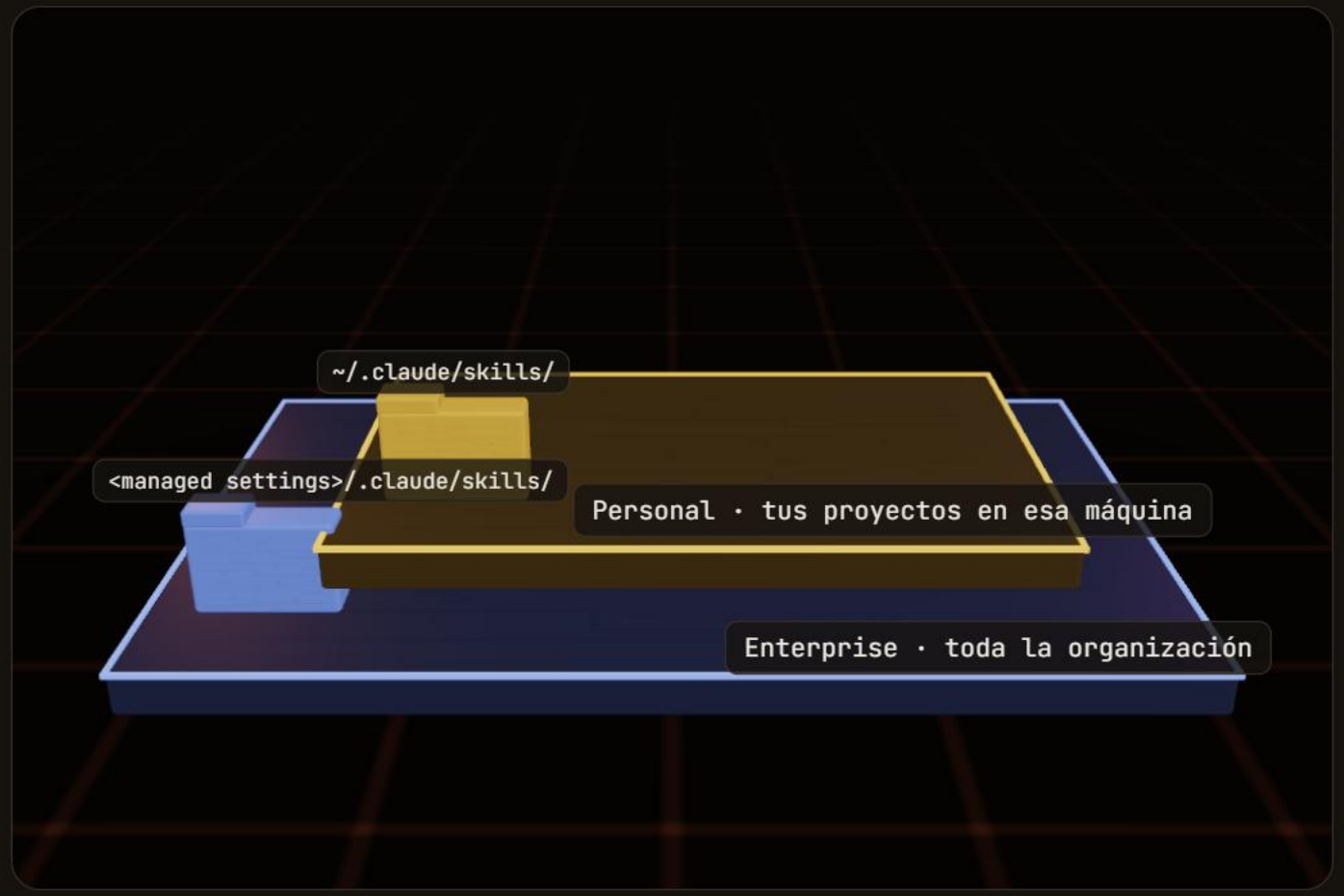

| NIVEL | RUTA | ALCANCE |
|---|---|---|
| Enterprise | <managed settings>/.claude/skills/<skill-name>/SKILL.md | Toda la organización<br>lo despliega un admin |
| Personal | ~/.claude/skills/<skill-name>/SKILL.md<br>Windows: C:/Users/<usuario>/.claude/skills | Todos tus proyectos<br>en esa máquina |
| Proyecto | .claude/skills/<skill-name>/SKILL.md | Ese repo<br>lo commiteas y tu equipo también lo tiene |

PASO ACTUAL

**Enterprise.** En el directorio de managed settings de Claude Code (macOS: /Library/Application Support/ClaudeCode/ · Linux: /etc/claude-code/ · Windows: C:\Program Files\ClaudeCode\). La despliega un admin y la ve toda la organización.

// 07 · Dónde se guardan

# Tres niveles, tres alcances: organización, tu máquina, el repo




| NIVEL | RUTA | ALCANCE |
|---|---|---|
| Enterprise | <managed settings>/.claude/skills/<skill-name>/SKILL.md | Toda la organización lo despliega un admin |
| Personal | ~/.claude/skills/<skill-name>/SKILL.md Windows: C:/Users/<usuario>/.claude/skills | Todos tus proyectos en esa máquina |
| Proyecto | .claude/skills/<skill-name>/SKILL.md | Ese repo lo commiteas y tu equipo también lo tiene |

PASO ACTUAL

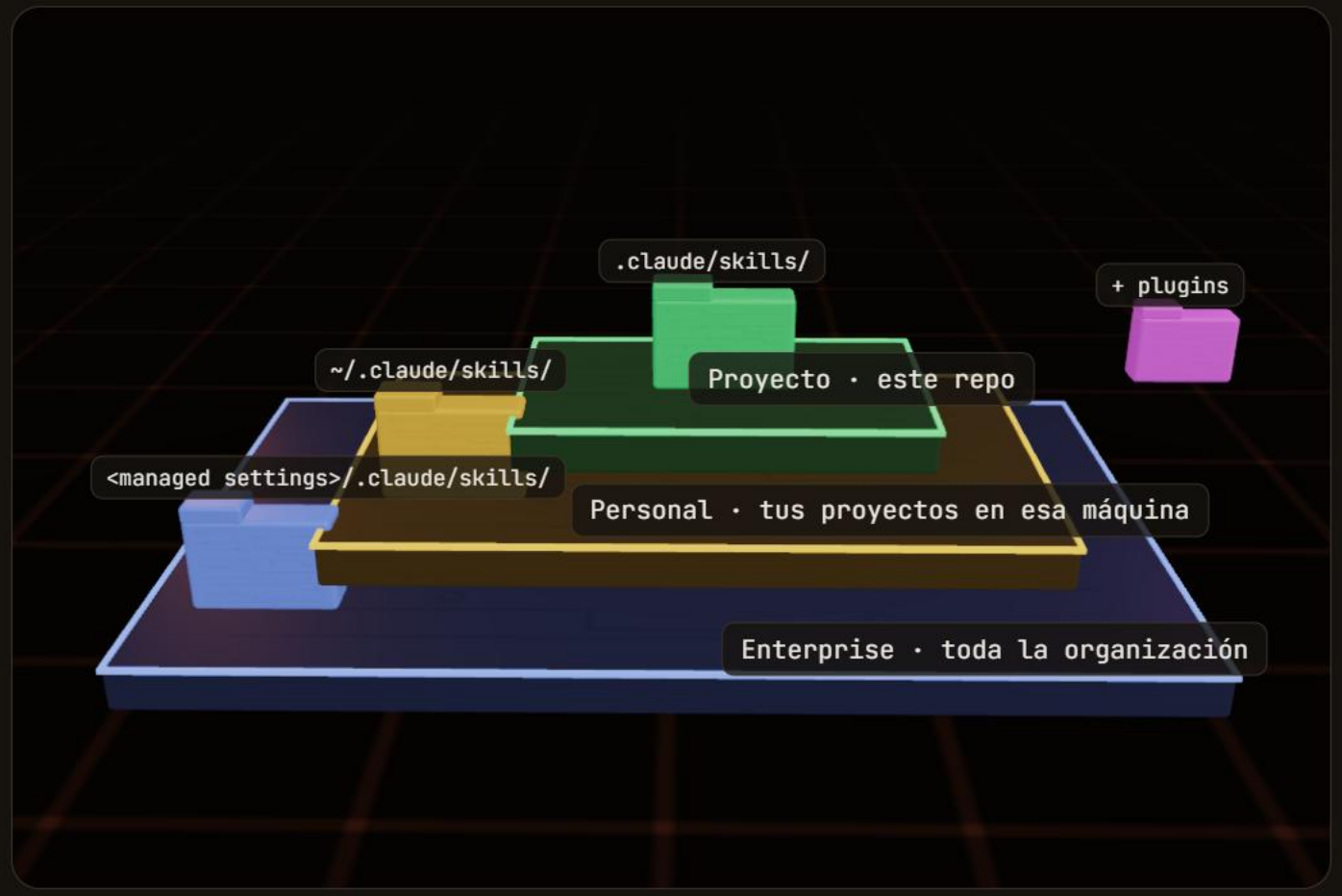
**Personal.** En ~/.claude/skills/ (Windows: C:/Users/<usuario>/.claude/skills). La tienes en todos tus proyectos de esa máquina, pero solo tú.



LOGO PÍLDORAS PENDIENTE

// 07 · Dónde se guardan

# Tres niveles, tres alcances: organización, tu máquina, el repo




| NIVEL | RUTA | ALCANCE |
|---|---|---|
| Enterprise | <managed settings>/.claude/skills/<skill-name>/SKILL.md | Toda la organización<br>lo despliega un admin |
| Personal | ~/.claude/skills/<skill-name>/SKILL.md<br>Windows: C:/Users/<usuario>/.claude/skills | Todos tus proyectos<br>en esa máquina |
| Proyecto | .claude/skills/<skill-name>/SKILL.md | Ese repo<br>lo commiteas y tu equipo también lo tiene |

PASO ACTUAL

**Proyecto.** En .claude/skills/ dentro del repo. La commiteas y tu equipo también la tiene. Además, hay una cuarta fuente: skills que vienen dentro de **plugins** instalados.



LOGO PÍLDORAS PENDIENTE

// 08 · Cómo se crea

# Un skill se crea en tres pasos

Haces una carpeta, escribes un `SKILL.md` adentro y, si quieres, agregas archivos de apoyo.

01

### Crear la carpeta

Con el nombre del Skill. Ese nombre será su identificador.

```
python-best-practices/
```

02

### Crear el archivo SKILL.md

Adentro de la carpeta, llamado exactamente así. Frontmatter + instrucciones.

```
python-best-practices/
└── SKILL.md
```

03

### Archivos adicionales opcional

Cuando el Skill crece: referencias, scripts y plantillas.

```
references/
scripts/
assets/
```

# Primero creas una carpeta con el nombre del Skill

bash · mi-proyecto

```
# dentro del repo (nivel Proyecto)
$ mkdir -p .claude/skills/python-best-practices
$ tree .claude/skills
.claude/skills
└── python-best-practices/

# o para todos tus proyectos (nivel Personal)
$ mkdir -p ~/.claude/skills/python-best-practices
```

### python-best-practices/

Una carpeta vacía con el nombre del Skill. Minúsculas y guiones: ese mismo nombre es el que luego escribes como `/python-best-practices`.

### ¿Dónde?

En `.claude/skills/` del repo si es para el equipo, o en `~/.claude/skills/` si es solo para ti (la tabla de la slide anterior).



Todavía no hay nada adentro: Claude aún no sabe que esta skill existe. Eso cambia en el paso 2.

# Dentro creas un archivo llamado exactamente SKILL.md

Este archivo tiene dos partes importantes.

python-best-practices/SKILL.md

```
---
name: python-best-practices
description: Aplica buenas prácticas de Python
  al desarrollar, revisar y refactorizar código.
---
```

```
# Python Best Practices

Cuando trabajes con Python:

## 1. Estilo de código
- Sigue las convenciones PEP 8.
- …
```

### Frontmatter

Metadatos que indican el **nombre** y **cuándo debe utilizarse** el Skill. Va entre los dos `---`.

### Instructions

Las instrucciones que Claude debe seguir. Markdown normal: títulos, listas, ejemplos.

El nombre importa: `SKILL.md` en mayúsculas. Es lo único obligatorio dentro de la carpeta.

// 11 · Paso 2 de 3 · el ejemplo completo

# ¡Y eso es todo! Ya tienes un Skill básico

python-best-practices/SKILL.md

```
---
name: python-best-practices
description: Aplica buenas prácticas de
  Python al desarrollar, revisar y
  refactorizar código.
---

# Python Best Practices

Cuando trabajes con Python:

## 1. Estilo de código
- Sigue las convenciones PEP 8.
- Utiliza nombres descriptivos.
- Evita duplicar código.

## 2. Tipado
- Utiliza Type Hints.
- Prefiere tipos explícitos cuando
  aporten claridad.

## 3. Calidad
- Utiliza pytest para las pruebas.
- Maneja correctamente las excepciones.
- Evita complejidad innecesaria.

## 4. Rendimiento
- Considera generadores para procesar
  grandes volúmenes de datos.
- Evita operaciones innecesarias
  en memoria.
```

**Frontmatter**
name + description

**4 secciones de instrucciones**
Estilo · Tipado · Calidad · Rendimiento

**Markdown normal**
Títulos y viñetas. Nada raro, nada que compilar.



Ya está instalado: Claude lo ve en la lista y lo cargará cuando revises o escribas Python.

// 12 · Paso 3 de 3 · opcional

# Cuando el Skill crece, puedes organizarlo mejor

tree python-best-practices

```
python-best-practices/
│
├── SKILL.md
│
├── references/
│   ├── pep8.md
│   └── typing.md
│
├── scripts/
│   └── check_code.py
│
└── assets/
    └── project_template/
```

| CARPETA | PROPÓSITO |
| --- | --- |
| references/ | Documentación y material de consulta. |
| scripts/ | Código ejecutable. |
| assets/ | Plantillas y recursos. |

No necesitas crear todas estas carpetas. **Depende de la complejidad del Skill.** Claude lee estos archivos solo cuando el SKILL.md los referencia y hacen falta (nivel 3 de la carga progresiva).

Recomendación oficial: mantén el `SKILL.md` por debajo de **500 líneas** y mueve el material detallado a estos archivos de apoyo.

# Skills enseñan cómo, MCP conecta, Hooks reaccionan

## Skills

Enseñan **cómo realizar una tarea**.

Conocimiento y procedimiento: "así se hace un reto de la academia", "así se revisa Python en este equipo". Viven en una carpeta con `SKILL.md`.

```
.claude/skills/retos-python/SKILL.md
```

## MCP

Conecta Claude con **herramientas y fuentes de datos externas**.

Bases de datos, APIs, servicios: le da a Claude manos y ojos fuera del repo, a través de un servidor MCP.

```
$ claude mcp add postgres …
```

## Hooks

Ejecutan acciones ante **determinados eventos**.

Antes o después de una herramienta, al terminar una respuesta: scripts automáticos que corren solos, como vimos en la entrega anterior.

```
PostToolUse → ruff check .
```

# Con esto finalizaríamos: una carpeta, un SKILL.md y Claude trabajando a tu manera.

carpeta + SKILL.md

description = cuándo

carga progresiva en 3 niveles

Enterprise · Personal · Proyecto

Skills ≠ MCP ≠ Hooks

